INFORMATICS INSTITUTE OF TECHNOLOGY

UNIVERSITY OF WESTMINSTER

5BUIS0170C – BIS Design and Architecture

`Individual Coursework Report

Task 3 – Patterns Individual Task Report

Module Leader: Janice Abeykoon

GROUP NAME: C7

STUDENT NAME: Sathalogithasiva Vyishnavan

IIT ID: 20221257

UoW ID: w2001515

Table of Contents

Contents

i. Table of Figures 2

1. Design Pattern Justification and Structure (300 Words) 3

i. Table of Figures

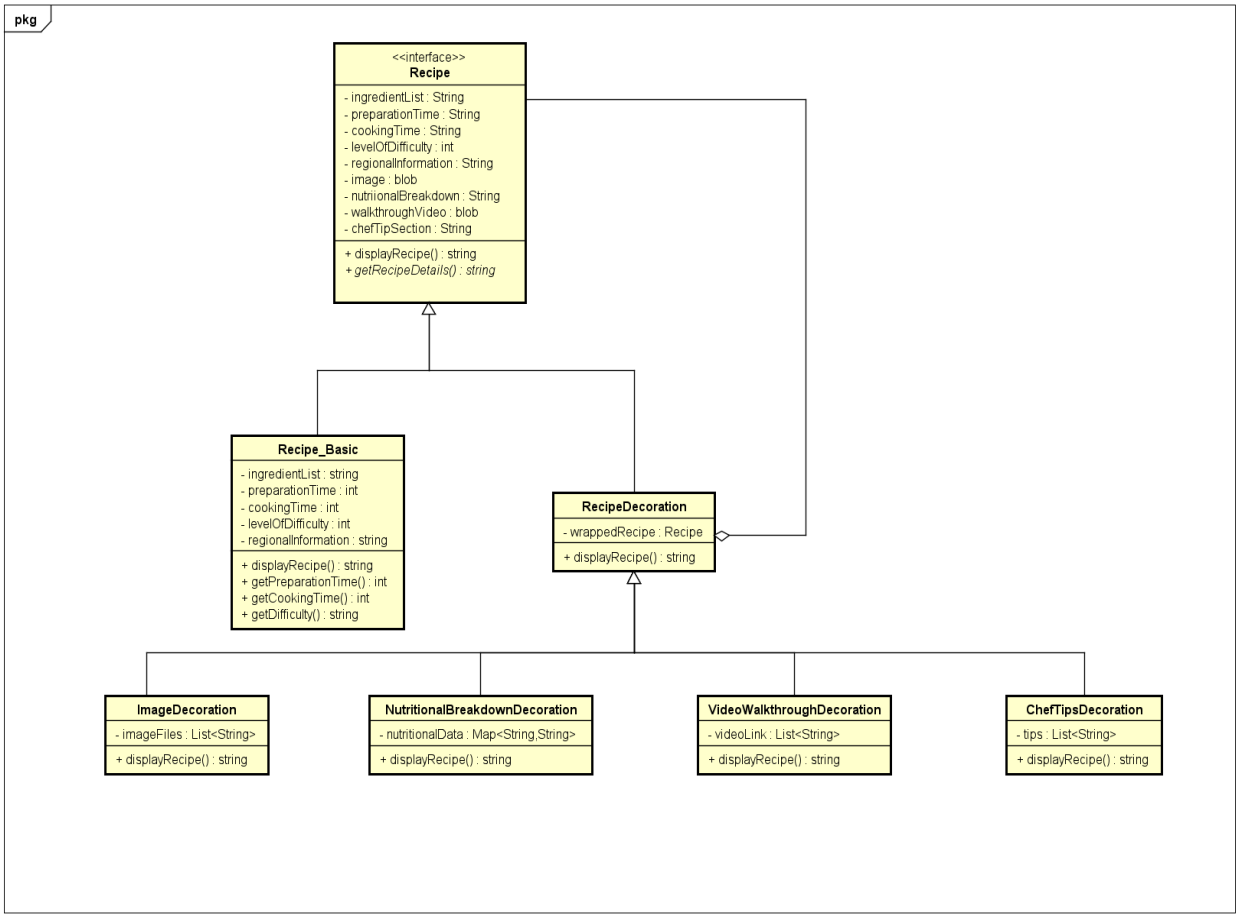

Figure 1:- Decoration pattern of Recipe Enhancement.

1. Design Pattern Justification and Structure (300 Words)

The decorator pattern is the most appropriate to use in the Recipe Enhancement case as it is used to provide optional features that are overlaid onto a base object. The ingredients list, the number of minutes it takes to prepare and cook, the level of difficulty, and regional information are implement in Recipe_Basic. On the other side Chef Tips, Images, Video Walkthrough and Nutritional Breakdown kind of features are optional. This pattern enables us to add these features to recipes dynamically during runtime. To realize all possible combinations of optional enhancements with inheritance alone it would be indefinite and would lead to the explosion of classes.

The Decoration pattern is a structural design pattern that adds extra responsibilities to an object seamlessly. It offers a flexible alternative to subclassing for extending functionality. It functions by defining a set of wrapper classes (Decorators) that implement the same interface as the object they wrap (Component). When a method is called on the decorator, it first forwards the call to the wrapped component and then adds its own behavior. We use this pattern when we need to give extra responsibilities to individual objects without changing their class structure and when we want a flexible, runtime way to combine different features onto a core object.

The diagram is designed to allow this composition: the <<interface >> Recipe (Component) defines the common displayRecipe() and getRecipeDetails() contracts. Recipe_Basic is the concreate component, containing the essential attributes and providing the base implementation. RecipeDecoration is an abstract Decorator, which keeps a reference to a Recipe object (wrappedRecipe). Finally, the Concrete Decorators (ImageDecoration, NutritionalBreakdownDecoration, VideoWalkthroughDecoration and ChefTipDecoration) wrap the component, adding their specific enhancement logic inside their displayRecipe() method. For example, ImageDecoration uses its imageFiles: List<String> attributes to add image display. The chain of responsibility ensures the final displayed recipe includes the basic details along with all desired optional enhancements.